OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID:119458

NQF Level: 6

Software Design & Testing with C#

SDT621

Formative Assessment 1

2026

CTU training solutions | Private Tertiary Education

Table of Contents

DECLARATION OF AUTHENTICITY

I Sfiso Skosana (FULL NAME)

hereby declare that the contents of this assessment Formative Assessment-1

is entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work accessed from any AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.

I have read and understood the Examination Rules and Regulations.

Signature: 

Date: 4/28/25

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing with C#
Module Code	SDT621
NQF Level	6
Assessment Type	Formative Assessment 1
Hand Out Date	13 / 0 4 / 2026
Hand in Date	28 / 04 / 2026
Total Marks	100
Assessment Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi
Internal Moderator	Michelle Du Toit
External Moderator	

Student Name and Surname	Sfiso Skosana
Student Number	20251311

Instructions:

Please read the following instructions carefully before attempting the assessment:

- Read each question carefully and consider the mark allocation before answering.
- Ensure that you answer all questions and provide clear, well-structured C# code and explanations where required.
- Apply your understanding of C# programming concepts to solve the problems based on the given scenarios.

Submission Requirements

- For your final submission, you must include:
- The Declaration of Authenticity
- Your completed Answer Sheet (PDF document)
- Your Visual Studio project files (compressed as a ZIP file)

Evidence Collection Requirements

- To support your submission, you must:
- Take a screenshot of the output for each question
- Ensure each screenshot shows the full screen, including the system date and time
- Do NOT submit screenshots of your source code
- Submit your complete project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- Push your project code to GitHub
- Ensure the repository contains the correct and complete solution
- Share the GitHub repository link in your submission for review and verification
- Example: https://github.com/your-username/project-name

Please ensure that your repository:

- builds successfully
- contains all project files
- matches the work submitted in your ZIP file

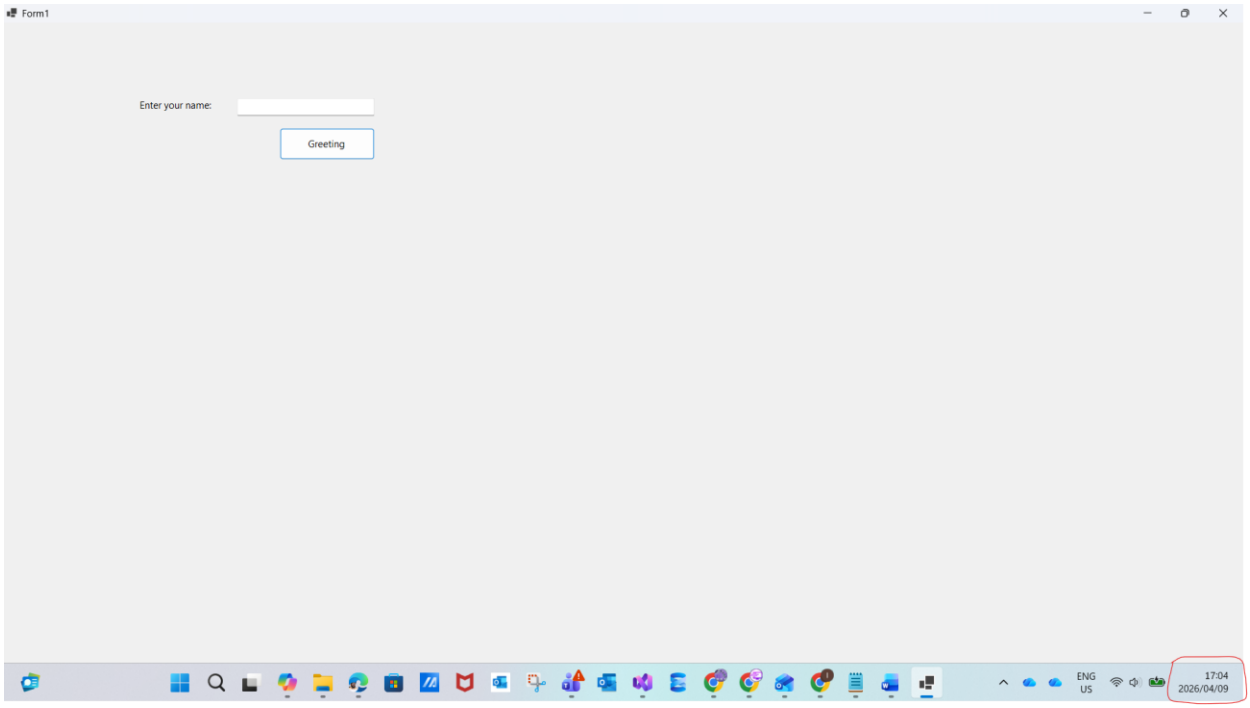
Assessment Outcomes:

- Create and run C# programs in Visual Studio.
- Declare and manipulate variables.
- Use operators and expressions.
- Understand scope (local, class-level, etc.).
- Write methods with parameters and return values.
- Create classes, objects, fields, properties, and constructors.

Example of Evidence Collection for Your Submission (PDF):

SECTION A: Short Answer Questions — 40 Marks

Question1 :

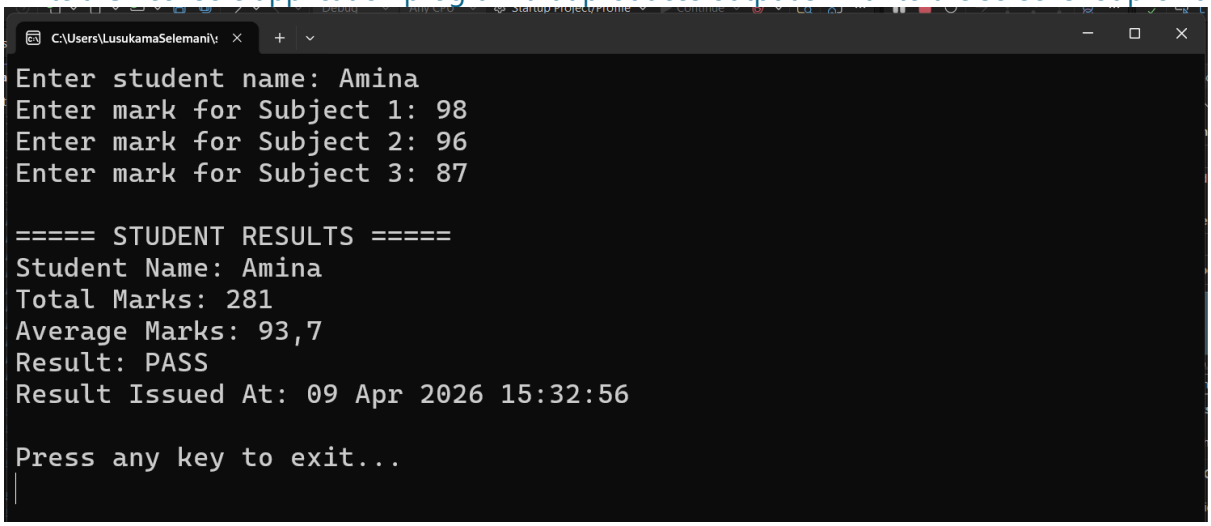


Ensure each screenshot shows the **full screen**, including the **system date and time**

SECTION A: Short Answer Questions. 40 Marks

Question 1 10 Marks

Write a C# console application program that produces output similar to the screenshot provided.

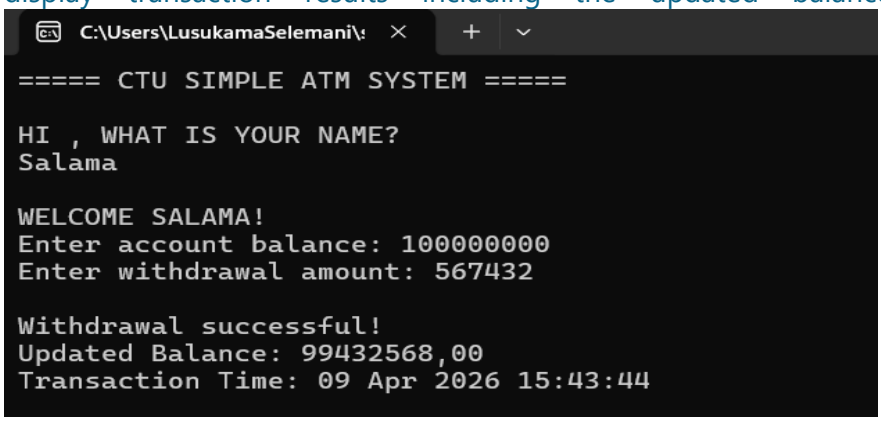


Your program must:

- Prompt the user to enter a student name
- Prompt the user to enter three subject marks
- Validate that marks entered are numeric values
- Calculate the total marks
- Calculate the average marks
- Display whether the student PASS or FAIL
- Rule: Average ≥ 50 = PASS
- Display results in the format shown above

Question 2 10 marks

Develop a Simple ATM Console Application using C# that simulates a basic banking withdrawal transaction. The system must interact with the user, accept inputs, perform calculations, and display transaction results including the updated balance and transaction time.

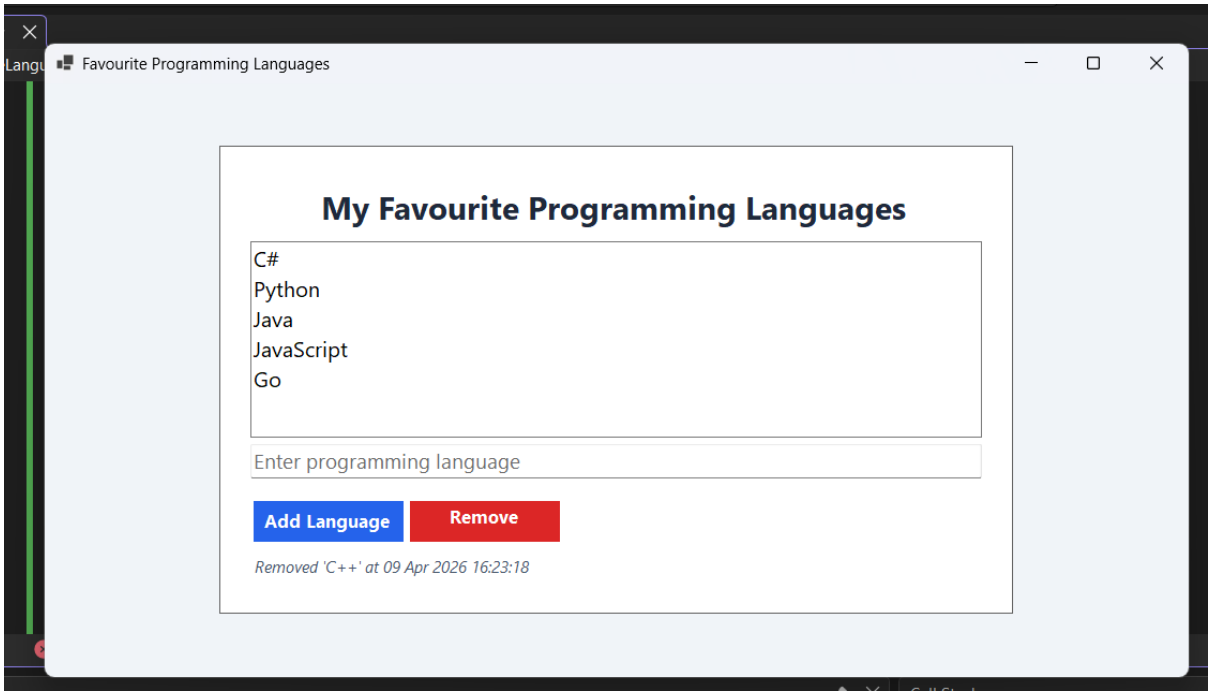


Question 3 20 Marks

Create a C# Windows Forms App application that performs the following functions:

- Allow the user to add a programming language to the list
- Prevent empty input from being added
- Prevent duplicate languages from being added
- Allow the user to remove a selected language
- Display the date and time when a language

Expected output:



SECTION B: Long Answer Questions. 50 Marks

Question 1: Emfuleni Municipality Console Application (20 Marks)

Scenario

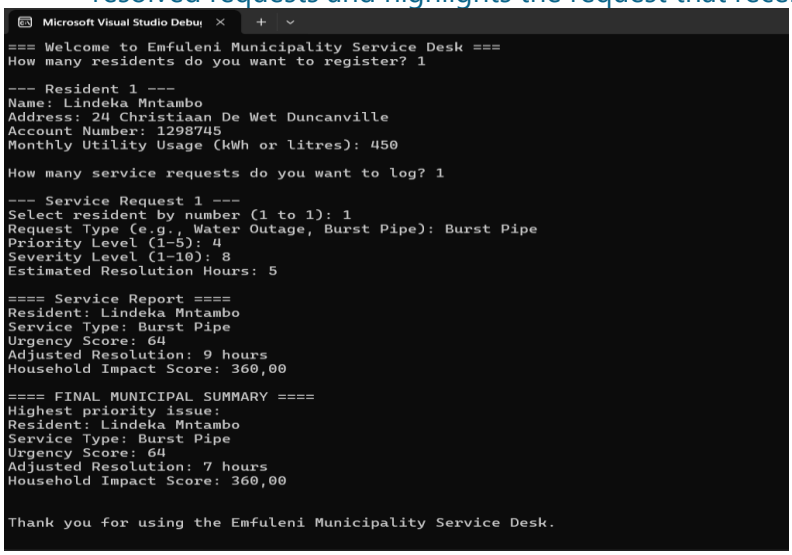
You are required to design and implement a console-based municipal service management application for Emfuleni Municipality. The purpose of this system is to enable the municipality to capture resident information, record utility-related service requests, and manage the processing of these requests interactively according to their priority and urgency levels.

Class Design

- **Resident.cs:** This class is responsible for storing resident information. The attributes include name, address, account number, and monthly utility usage.
- **ServiceRequest.cs:** This class captures and stores details about service requests. The attributes include request type, priority level, severity level, and estimated resolution time.
- **UtilitiesManager.cs:** This class processes service requests. Its responsibilities include calculating urgency scores for requests and generating comprehensive service reports.

Main Program Workflow (Program.cs)

1. The system prompts the user to enter the number of residents and gathers each resident's details.
2. Next, the user is asked to specify the number of service requests, after which the system collects information for each request. This includes associating the request with a specific resident, capturing the request type, priority level (ranging from 1 to 5), severity level (ranging from 1 to 10), and estimated resolution hours.
3. The application displays a queue of pending service requests, each accompanied by its urgency score.
4. The user can interactively select requests from the queue to process.
5. For each processed request, the application generates a report that includes resident details, service request details, and the calculated urgency score.
6. After all requests have been processed, a summary is shown. This summary lists all resolved requests and highlights the request that received the highest urgency score.



Question 2: Digital Identity Processor – Windows Forms Application 30 Marks

Application Overview

The Windows Forms Application, named **HomeAffairsDigitalIdentityProcessor**, is designed to facilitate the creation and validation of digital citizen profiles. The application enables users to input a person's details, validate their identification number, and generate a summary profile in a graphical user interface.

Graphical User Interface Components

- **Labels:** Used for guiding the user through the entry of name, ID number, citizenship selection, and for displaying results.
- **TextBoxes:** Two text boxes are provided; one for the user's name and one for the ID number.
- **ComboBox:** This control allows the user to select citizenship status from at least three options: Citizen, Permanent Resident, and Visitor.
- **Buttons:** There are two buttons—*Validate ID* and *Generate Profile*—to perform the respective actions.
- **Results Display:** The output summary is shown in a multi-line TextBox or Label, presenting the processed information.
- CitizenProfile Class Design
- The **CitizenProfile** class holds critical information for each profile and encapsulates the logic required for validation and age calculation.

Properties:

- FullName
- IDNumber
- Age
- CitizenshipStatus
- **Constructor:** Accepts the name, ID number, and citizenship status, then calculates the age using a dedicated method.
- **Age Calculation:** The method extracts the birth date from the first six digits of the ID number (YYMMDD), determines the century (1900s or 2000s), and computes the current age accordingly.

ValidateID() Method:

- Ensures the ID number contains exactly 13 digits.
- Verifies that the ID number is fully numeric.
- Uses the calculated age as part of the validation criteria.
- Returns a validation message indicating the result of these checks.

Application Functionality

- When the **Validate ID** button is pressed, the application reads the user inputs, creates a *CitizenProfile* object, and displays the validation result in the results area.
- Pressing the **Generate Profile** button produces a formatted profile summary containing the full name, ID number, calculated age, citizenship status, validation result, and a processing timestamp message.

Expected output

External Evidence — 10 Marks

GitHub link and zip file submitted with PDF answer.

Missing link will result in losing 10 marks.

TOTAL MARKS _____ / 100

End of Assessment

SECTION A RUBRIC (SHORT PRACTICAL QUESTIONS) – 40 MARKS

Criteria	Excellent (Full Competency)	Competent (Partial Competency)	Needs Improvement	Marks
User Input Handling	Accepts and validates all inputs correctly	Accepts inputs but limited validation	Missing validation or incorrect input logic	5
Console Interaction Flow	Logical prompts and structured execution	Minor flow issues	Poor interaction structure	5
Calculation Logic	Accurate totals / averages / balances	Minor calculation errors	Incorrect calculations	5
Decision Structures	Correct PASS/FAIL and withdrawal logic	Minor logic errors	Missing or incorrect decisions	5
Output Formatting	Output matches required structure clearly	Minor formatting issues	Poor formatting	5
Windows Forms GUI Controls	Controls correctly implemented	Some missing controls	Incorrect GUI implementation	5
Event Handling	All events functional	Partial functionality	Events missing or incorrect	5
Validation & Defensive Programming	Prevents duplicates / empty input / invalid data	Partial validation	No validation	5

Section A Total = _____ / 40 Marks

SECTION B RUBRIC (LONG PRACTICAL QUESTIONS) – 50 MARKS

QUESTION 1 RUBRIC

Emfuleni Municipality Console Application (20 Marks)

Criteria	Excellent	Competent	Needs Improvement	Marks

Resident Class Implementation	All attributes correct with constructor & usage	Minor omissions	Class poorly structured	3
ServiceRequest Class Implementation	Fully captures request properties correctly	Minor attribute issues	Incorrect class design	3
UtilitiesManager Class Logic	Correct urgency calculation + reporting workflow	Partial processing logic	Missing manager logic	4
Resident Data Capture Workflow	Captures residents dynamically	Minor interaction issues	Static / incomplete capture	2
Service Request Capture Workflow	Captures requests correctly with associations	Minor linkage issues	Incorrect request handling	2
Priority + Severity Validation	Correct numeric validation ranges enforced	Partial validation	No validation	2
Queue Display with Urgency Scores	Displays structured queue correctly	Minor display issues	Queue missing	1
Interactive Processing Simulation	Allows request processing selection	Limited interaction	Missing interaction	1
Generated Processing Report	Report includes resident + request + urgency score	Partial report output	Report incomplete	1
Final Summary Report	Lists resolved requests + highest urgency	Partial summary	Missing summary	1

Question 1 Total = _____ / 20 Marks

QUESTION 2 RUBRIC

Digital Identity Processor – Windows Forms Application (30 Marks)

Criteria	Excellent	Competent	Needs Improvement	Marks
GUI Layout Structure	All controls implemented correctly	Minor missing components	Incorrect layout	4
Label Usage & Guidance Structure	Labels clearly guide user workflow	Minor clarity issues	Poor UI guidance	2
TextBox Inputs (Name + ID)	Inputs captured correctly	Minor validation issues	Incorrect capture	2

ComboBox Citizenship Selection	Correct implementation with required options	Minor option issue	Incorrect setup	2
Validate ID Button Logic	Executes validation workflow correctly	Partial validation	Incorrect logic	4
CitizenProfile Class Design	Proper encapsulation with properties + constructor	Minor class issues	Incorrect structure	4
Age Calculation Logic (YYMMDD extraction)	Correct century detection & age calculation	Minor logic issues	Incorrect age logic	4
ValidateID() Method Implementation	Checks numeric + 13 digits + logical validation	Partial checks	Missing validation	3
Generate Profile Button Logic	Generates structured formatted profile summary	Partial formatting	Missing summary output	3
Timestamp Display	Timestamp correctly displayed	Minor formatting issue	Missing timestamp	1
Results Display Output	Clear structured multiline output	Minor formatting issues	Poor output formatting	1

Question 2 Total = _____ / 30 Marks

EXTERNAL EVIDENCE RUBRIC – 10 MARKS

This supports authenticity verification and workplace readiness expectations

Criteria	Excellent	Competent	Needs Improvement	Marks
GitHub Repository Submitted	Repository accessible with full solution	Repository incomplete	Missing repository	5
Zip File Submitted	Full working solution included	Partial solution included	Missing zip file	5

External Evidence Total = _____ / 10 Marks

MARK ALLOCATION

Section	Description	Marks
Section A	Short Programming Tasks	_____ / 40
Section B Question 1	Municipality Console Application	_____ / 20
Section B Question 2	Windows Forms Identity Processor	_____ / 30
External Evidence	GitHub + Zip Submission	_____ / 10
TOTAL		_____ / 100 Marks

LEARNING OUTCOMES CROSS-MAPPING

Learning Outcome	Description	Question Alignment	Evidence Produced
LO1	Apply variables and data types	Section A, B	Input capture
LO2	Implement decision structures	Section A, B	PASS/FAIL, validation
LO3	Perform numeric validation	Section A, B	Range checking
LO4	Develop console applications	Section A, B Q1	Workflow simulation
LO5	Implement object-oriented classes	Section B Q1 & Q2	Resident + Profile classes
LO6	Apply encapsulation	Section B	CitizenProfile
LO7	Implement event-driven programming	Section A Q3 + Section B Q2	Button actions
LO8	Perform data validation logic	Section A + B	Duplicate prevention
LO9	Implement structured reports	Section B	Service summary
LO10	Apply GUI development principles	Section A Q3 + Section B Q2	WinForms design
LO11	Apply algorithmic workflow modelling	Section B Q1	Queue processing
LO12	Implement defensive programming	Section A + B	Input validation